



상하이 링쿵 테크놀로지

모터 CAN 버스 통신 프로토콜

V2.36

목차

상하이 링콩 테크놀로지	1
모터 CAN 버스 통신 프로토콜	1
면책 조항	4
CAN 버스 매개변수	5
단일 모터 명령	5
1. 모터 상태 1 및 오류 플래그 읽기 명령	5
2. 모터 오류 플래그 지우기 명령	6
3. 모터 상태 2 읽기 명령	6
4. 모터 상태 3 읽기 명령	7
5. 모터 정지 명령	8
6. 모터 작동 명령	8
7. 모터 정지 명령	8
8. 브레이크 제어 및 상태 읽기 명령	8
9. 개루프 제어 명령(이 명령은 MS 모터에서만 구현되며, 다른 모터에서는 무효)	9
10. 토크 페루프 제어 명령 (이 명령은 MF, MH, MG 모터에서만 구현됨)	9
11. 속도 폐쇄 루프 제어 명령 1	오류! 정의되지 않은 북마크입니다.
12. 속도 폐쇄 루프 제어 명령 2	10
13. 다중 회전 위치 폐쇄 루프 제어 명령 1	10
14. 다회전 위치 폐쇄 루프 제어 명령 2	11
15. 단일 회전 위치 폐쇄 루프 제어 명령 1	11
16. 단일 궤도 위치 폐쇄 루프 제어 명령 2	12
17. 증분 위치 폐쇄 루프 제어 명령 1	12
18. 증분 위치 폐쇄 루프 제어 명령 2	13
19. 제어 매개변수 읽기 명령	13
20. 쓰기 제어 매개변수 명령	14
21. 모터 인코더 데이터 읽기 명령	14
22. 현재 위치를 ROM 에 모터 제로점으로 설정 명령	15
23. 다중 회전 각도 읽기 명령	15
24. 단일 회전 각도 명령 읽기	16
25. 현재 위치를 임의의 각도로 설정(RAM 에 쓰기)	16
부록 1: 모터 제어 매개변수 표	17

면책 조항

상하이 링쿤 테크놀로지 유한공사 모터 드라이브 일체형 제어 시스템을 구매해 주셔서 감사합니다. 사용 전 본 고지사항을 주의 깊게 읽어주시기 바랍니다. 사용 시 본 고지사항의 모든 내용에 대한 동의 및 수락으로 간주됩니다. 본 제품은 제품 설명서, 제어 프로토콜 및 관련 법규, 정책, 기준을 엄격히 준수하여 설치 및 사용해야 합니다. 제품 사용 과정에서 사용자는 자신의 행동 및 이에 따른 모든 결과에 대한 책임을 지는 것으로 간주됩니다. 사용자의 부적절한 사용, 설치, 개조로 인한 모든 손실에 대해 링쿤 테크놀로지는 법적 책임을 지지 않습니다.

링쿤 테크놀로지는 상하이 링쿤 테크놀로지 유한공사 및 그 계열사의 상표입니다. 본문에 등장하는 제품명, 브랜드 등은 해당 소속 회사의 상표 또는 등록상표입니다.

본 제품 및 매뉴얼의 저작권은 상하이 링쿤 테크놀로지 유한공사에 있습니다. 허가 없이 어떠한 형태로도 복제 또는 재인쇄할 수 없습니다. 면책 조항에 대한 최종 해석권은 당사에 있습니다.

CAN 버스 파라미터

버스 인터페이스: CAN

보트레이트(일반 모드, 단일 모터 명령):

1Mbps(기본값)
500kbps
250kbps
125kbps
100kbps

보트 속도(브로드캐스트 모드, 다중 모터 명령):

1Mbps 500kbps

단일 모터 명령

동일한 버스에는 최대 32개(버스 부하 상황에 따라 다름)의 드라이버를 장착할 수 있으며, 버스 충돌을 방지하기 위해 각 드라이버는 서로 다른 ID를 설정해야 합니다.

마스터가 버스로 단일 모터 명령을 전송하면, 해당 ID의 모터가 명령 수신 후 실행하고 일정 시간(0.25ms 이내) 내에 마스터에게 응답을 전송합니다. 명령 패킷과 응답 패킷 형식은 다음과 같습니다:

명령 메시지 식별자: 0x140 + ID(1~32) 등

답 메시지 식별자: 0x180 + ID(1~32) 프레

임 형식: 데이터 프레임

프레임 유형: 표준 프레임

DLC: 8바이트

1. 모터 상태 1 및 오류 플래그 읽기 명령

이 명령은 현재 모터의 온도, 전압 및 오류 상태 플래그를 읽습니다

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9A
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 매개변수가 포함됩니다:

1. 모터 온도 temperature(int8_t 유형, 단위 1°C/LSB).
2. 버스 전압 voltage(int16_t 유형, 단위 0.01V/LSB).
3. 버스 전류 current(int16_t 유형, 단위 0.01A/LSB).
4. 모터 상태 motorState(uint8_t 유형, 각 비트가 서로 다른 모터 상태를 나타냄)
5. 오류 상태 errorState(uint8_t 유형, 각 비트가 서로 다른 모터 오류 상태를 나타냄)

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9A
DATA[1]	모터 온도	DATA[1] = *(uint8_t *)&temperature
DATA[2]	버스 전압 하위 바이트	DATA[2] = *(uint8_t *)&voltage

DATA[3]	버스 전압 상위 바이트	DATA[3] *((uint8_t *)&voltage)+1
DATA[4]	버스 전류 하위 바이트	DATA[4] = *((uint8_t *)¤t)
DATA[5]	버스 전류 상위 바이트	DATA[5] = *((uint8_t *)¤t)+1
DATA[6]	모터 상태 바이트	DATA[6] = motorState
DATA[7]	오류 상태 바이트	DATA[7] = errorState

참고:

1. motorState = 0x00 모터가 켜진 상태; motorState = 0x10 모터가 꺼진 상태.
2. errorState 각 비트의 구체적인 상태는 다음과 같습니다

errorState 비트	상태 설명	0	1
0	저전압 상태	정상	저전압 보호
1	고전압 상태	정상	고압 보호
2	구동 온도 상태	정상	구동 과열
3	모터 온도 상태	정상	모터 과열
4	모터 전류 상태	정상	모터 과전류
5	모터 단락 상태	정상	모터 단락
6	정지 상태	정상	모터 정지
7	입력 신호 상태	정상	입력 신호 손실 시간 초과

2. 모터 오류 플래그 지우기 명령

이 명령은 현재 모터의 오류 상태를 지우며, 모터가 수신 후 응답합니다

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9B
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답합니다. 응답 데이터는 모터 상태 1 및 오류 플래그 읽기 명령과 동일합니다(단, 명령

바이트 DATA[0]만 다름

다름, 여기서는 0x9B) 참

고:

1. 모터 상태가 정상으로 복구되지 않으면 오류 플래그를 지울 수 없습니다.

3. 모터 상태 2 읽기 명령

이 명령은 현재 모터의 온도, 모터 토크 전류(MF, MG)/모터 출력 전력(MS), 회전 속도, 인코더 위치를 읽습니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9C
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 매개변수가 포함됩니다.

1. 모터 온도 temperature(int8_t 유형, 1°C/LSB).
2. MF, MG 모터의 토크 전류 값 iq 또는 MS 모터의 출력 전력 값 power, int16_t 유형. MG 모터 iq 분해능은 (66/4096 A) / LSB; MF 모터 iq 분해능은 (33/4096 A) / LSB. MS 모터 power 범위는 -1000~1000.
3. 모터 속도 speed(int16_t 유형, 1dps/LSB).
4. 인코더 값 encoder(uint16_t 타입, 14비트 인코더의 수치 범위 0~16383, 15비트 인코더의 수치 범위 0~32767, 16비트 인코더의 수치 범위 0~65535).

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9C
DATA[1]	모터 온도	DATA[1] = *(uint8_t *)&temperature
DATA[2]	토크 전류 하위 바이트	DATA[2] = *(uint8_t *)&iq
	출력 전력 하위 바이트(MS 시리즈)	DATA[2] = *(uint8_t *)&power
DATA[3]	토크 전류 상위 바이트	DATA[3] = *((uint8_t *)&iq)+1
	출력 전력 상위 바이트(MS 시리즈)	DATA[3] = *((uint8_t *)&power)+1
DATA[4]	모터 속도 하위 바이트	DATA[4] = *(uint8_t *)&speed
DATA[5]	모터 속도 상위 바이트	DATA[5] = *((uint8_t *)&speed)+1
DATA[6]	인코더 위치 하위 바이트	DATA[6] = *(uint8_t *)&encoder
DATA[7]	인코더 위치 상위 바이트	DATA[7] = *((uint8_t *)&encoder)+1

4. 모터 상태 3 명령어 읽기
MS 모터에는 상 전류 샘플링 기능이 없으므로, 이 명령은 MS 모터에서 작동하지 않습니다. 이 명령은 현재 모터의 온도와 3상 전류 데이터를 읽습니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9D
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 정보가 포함됩니다:

1. 모터 온도 temperature(int8_t 유형, 1°C/LSB)
2. 상 전류 데이터 iA, iB, iC, 데이터 유형은 int16_t 유형, MG 모터 상 전류 분해능은 (66/4096 A) / LSB; MF 모터 상전류 분해능은 (33/4096 A) / LSB입니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x9D
DATA[1]	모터 온도	DATA[1] = *(uint8_t *)&temperature
DATA[2]	A상 전류 하위 바이트	DATA[2] = *(uint8_t *)&iA
DATA[3]	A상 전류 상위 바이트	DATA[3] = *((uint8_t *)&iA)+1
DATA[4]	B상 전류 하위 바이트	DATA[4] = *(uint8_t *)&iB
DATA[5]	B상 전류 상위 바이트	DATA[5] = *((uint8_t *)&iB)+1
DATA[6]	C상 전류 하위 바이트	DATA[6] = *(uint8_t *)&iC
DATA[7]	C상 전류 상위 바이트	DATA[7] = *((uint8_t *)&iC)+1

5. 모터 정지 명령

모터를 켜진 상태(전원 공급 후 기본 상태)에서 꺼진 상태로 전환하고, 모터 회전 횟수 및 이전에 수신한 제어 명령을 초기화합니다.

LED는 상시 점등에서 느린 깜빡임으로 전환됩니다. 이때 모터는 여전히 제어 명령을 수신할 수 있으나 동작을 수행하지 않습니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x80
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

호스트와 동일한 데이터를 전송합니다.

6. 모터 작동 명령

모터를 꺼진 상태에서 켜진 상태로 전환하며, LED는 느린 깜빡임에서 계속 켜진 상태로 바뀝니다. 이때 제어 명령을 전송하면

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x88
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

호스트와 동일한 명령을 전송합니다.

7. 모터 정지 명령

모터를 정지시키지만 모터 작동 상태는 지우지 않습니다. 다시 제어 명령을 전송하면 모터 동작을 제어할 수 있습니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x81
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답 (1 프레임)

호스트와 동일한 데이터를 전송합니다.

8. 브레이크 제어 및 상태 읽기 명령

브레이크 개폐 제어 또는 현재 브레이크 상태 읽기.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x8C

DATA[1]	브레이크 상태 제어 및 읽기 바이트	0x00: 브레이크 전원 차단, 브레이크 작동 0x01: 브레이크 장치 전원 공급, 브레이크 해제 0x10: 브레이크 상태 읽기
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답 (1 프레임)

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x8C
DATA[1]	브레이크 상태 바이트	0x00: 브레이크가 전원 차단 상태이며, 브레이크 작동 0x01: 브레이크가 전원 공급 상태이며, 브레이크가 해제됨
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

9. 개루프 제어 명령 (이 명령은 **MS** 모터에서만 구현되며, 다른 모터에서는 무효)

호스트는 이 명령을 전송하여 모터로 출력되는 개루프 전압을 제어합니다. 제어값 **powerControl**은 **int16_t** 유형이며, 수치 범위는 -850~850입니다(모터 전류 및 토크는 모터에 따라 다름).

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA0
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	개루프 제어값 하위 바이트	DATA[4] = *((uint8_t *)&powerControl)
DATA[5]	개루프 제어값 상위 바이트	DATA[5] = *((uint8_t *)&powerControl)+1
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

비고:

1. 이 명령의 제어값 **powerControl**은 상위 컴퓨터의 Max Power 값 제한을 받지 않습니다. 드라이버 응답 (1 프레임)

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다름, 여기서는 0xA0).

10. 토크 폐쇄 루프 제어 명령 (이 명령은 **MF**, **MH**, **MG** 모터에서만 구현됨)

호스트는 이 명령을 전송하여 모터의 토크 전류 출력을 제어합니다. 제어값 **iqControl**은 **int16_t** 유형이며, 수치 범위는 -2048~2048입니다. 이는 MF 모터의 실제 토크 전류 범위 -16.5A~16.5A에 해당하며, MG 모터의 실제 토크 전류 범위 -33A~33A에 해당합니다. 버스 전류와 모터의 실제 토크는 모터 유형에 따라 다릅니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA1
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00

DATA[3]	NULL	0x00
DATA[4]	토크 전류 제어값 하위 바이트	DATA[4] = *(uint8_t *)&iqControl
DATA[5]	토크 전류 제어값 상위 바이트	DATA[5] = *((uint8_t *)&iqControl)+1
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

비고:

- 이 명령의 제어값 iqControl은 상위 컴퓨터의 최대 토크 전류 값에 제한받지 않습니다. 드라이버 응답
모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다름, 여기서는 0xA1).

11. 속도 폐쇄 제어 명령

호스트는 이 명령을 전송하여 모터 속도를 제어하며, 동시에 토크 제한을 적용합니다. 제어값 speedControl은 int32_t 유형이며, 실제 회전 속도는 0.01dps/LSB에 해당합니다. 제어값 iqControl은 int16_t 유형이며, 수치 범위는 -2048~2048입니다. 이는 MF 모터의 실제 토크 전류 범위 -16.5A~16.5A에 해당하며, MG 모터의 경우 실제 토크 전류 범위 -33A~33A에 해당합니다. 버스 전류와 모터의 실제 토크는 모터 유형에 따라 다릅니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA2
DATA[1]	NULL	0x00
DATA[2]	토크 전류 제어값 하위 바이트	DATA[2] = *(uint8_t *)&iqControl
DATA[3]	토크 전류 제어값 상위 바이트	DATA[3] = *((uint8_t *)&iqControl)+1
DATA[4]	속도 제어 하위 바이트	DATA[4] = *(uint8_t *)&speedControl
DATA[5]	속도 제어	DATA[5] = *((uint8_t *)&speedControl)+1
DATA[6]	속도 제어	DATA[6] = *((uint8_t *)&speedControl)+2
DATA[7]	속도 제어 상위 바이트	DATA[7] = *((uint8_t *)&speedControl)+3

비고:

- 이 명령에서 모터의 speedControl은 상위 컴퓨터의 Max Speed 값에 의해 제한됩니다.
- 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값으로 제한됩니다. 드라이버 응답
모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다름, 여기서는 0xA2).

12. 다중 회전 위치 폐쇄 루프 제어 명령 1

호스트는 이 명령을 전송하여 모터의 위치(다회전 각도)를 제어합니다. 제어값 angleControl은 int32_t 유형이며, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 나타내며, 모터 회전 방향은 목표 위치와 현재 위치의 차이에 의해 결정됩니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA3
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	위치 제어 하위 바이트	DATA[4] = *(uint8_t *)&angleControl
DATA[5]	위치 제어	DATA[5] = *((uint8_t *)&angleControl)+1
DATA[6]	위치 제어	DATA[6] = *((uint8_t *)&angleControl)+2
DATA[7]	위치 제어 상위 바이트	DATA[7] = *((uint8_t *)&angleControl)+3

비고:

- 이 명령어의 제어값 angleControl은 상위 컴퓨터의 Max Angle 값에 의해 제한됩니다.
- 이 명령어에서 모터의 최대 속도는 상위 컴퓨터의 Max Speed 값에 의해 제한됩니다.

- 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.
- 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값에 의해 제한됩니다;
MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값에 의해 제한됩니다.

드라이버 응답

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA3입니다).

13. 다회전 위치 폐쇄 제어 명령 2

호스트는 이 명령을 전송하여 모터의 위치(다회전 각도)를 제어합니다.

- 제어값 angleControl은 int32_t 유형이며, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 나타내며, 모터 회전 방향은 목표 위치와 현재 위치의 차이에 의해 결정됩니다.
- 제어값 maxSpeed는 모터 회전의 최대 속도를 제한하며, uint16_t 유형으로 실제 회전 속도 1dps/LSB에 해당합니다. 즉, 360
는 360dps를 나타냅니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA4
DATA[1]	NULL	0x00
DATA[2]	속도 제한 하위 바이트	DATA[2] = *(uint8_t *)&maxSpeed
DATA[3]	속도 제한 상위 바이트	DATA[3] = *((uint8_t *)&maxSpeed)+1
DATA[4]	위치 제어 하위 바이트	DATA[4] = *(uint8_t *)&angleControl
DATA[5]	위치 제어	DATA[5] = *((uint8_t *)&angleControl)+1
DATA[6]	위치 제어	DATA[6] = *((uint8_t *)&angleControl)+2
DATA[7]	위치 제어 상위 바이트	DATA[7] = *((uint8_t *)&angleControl)+3

비고:

- 이 명령어의 제어값 angleControl은 상위 컴퓨터의 Max Angle 값에 의해 제한됩니다.
- 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.
- 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값에 의해 제한됩니다;
MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값으로 제한됩니다.

드라이버 응답 (1 프레임)

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA4입니다).

14. 단일 회전 위치 폐쇄 루프 제어 명령 1

호스트는 이 명령을 전송하여 모터의 위치(단일 회전 각도)를 제어합니다.

- 제어값 spinDirection은 모터 회전 방향을 설정하며, uint8_t 타입입니다. 0x00은 시계 방향, 0x01은 반시계 방향을 나타냅니다.
- 제어값 angleControl은 uint32_t 타입이며, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 나타냅니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA5
DATA[1]	회전 방향 바이트	DATA[1] = spinDirection
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	위치 제어 바이트 1 (bit0 : bit7)	DATA[4] = *(uint8_t *)&angleControl
DATA[5]	위치 제어 바이트 2 (bit8 : bit15)	DATA[5] = *((uint8_t *)&angleControl)+1
DATA[6]	위치 제어 바이트 3 (bit16 : bit23)	DATA[6] = *((uint8_t *)&angleControl)+2
DATA[7]	위치 제어 바이트 4 (bit24 : bit31)	DATA[7] = *((uint8_t *)&angleControl)+3

비고:

- 이 명령에서 모터의 최대 속도는 상위 컴퓨터의 Max Speed 값에 의해 제한됩니다.
- 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.

- 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값으로 제한됩니다; MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값으로 제한됩니다.

드라이버 응답

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA5입니다).

15. 단일 회전 위치 펄스 제어 명령 2

호스트는 이 명령을 전송하여 모터의 위치(단일 회전 각도)를 제어합니다.

- 제어값 spinDirection은 모터 회전 방향을 설정하며, uint8_t 타입입니다. 0x00은 시계 방향, 0x01은 반시계 방향을 나타냅니다.
- angleControl은 uint32_t 타입으로, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 의미합니다.
- 속도 제어값 maxSpeed는 모터 회전의 최대 속도를 제한하며, uint16_t 타입입니다. 실제 회전 속도는 1dps/LSB에 해당하며, 즉 360은 360dps를 의미합니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA6
DATA[1]	회전 방향 바이트	DATA[1] = spinDirection
DATA[2]	속도 제한 바이트 1 (bit0 : bit7)	DATA[2] = *(uint8_t *)&maxSpeed
DATA[3]	속도 제한 바이트 2 (bit8 : bit15)	DATA[3] = *((uint8_t *)&maxSpeed)+1
DATA[4]	위치 제어 바이트 1 (bit0 : bit7)	DATA[4] = *(uint8_t *)&angleControl
DATA[5]	위치 제어 바이트 2 (bit8 : bit15)	DATA[5] = *((uint8_t *)&angleControl)+1
DATA[6]	위치 제어 바이트 3 (bit16 : bit23)	DATA[6] = *((uint8_t *)&angleControl)+2
DATA[7]	위치 제어 바이트 4 (bit24 : bit31)	DATA[7] = *((uint8_t *)&angleControl)+3

비고:

- 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.
 - 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값에 의해 제한됩니다; MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값으로 제한됩니다.
- 드라이버 응답 (1 프레임)
- 모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 읽기 2 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA6입니다).

16. 증분 위치 펄스 제어 명령 1

호스트는 이 명령을 전송하여 모터의 위치 증분을 제어합니다.

제어값 angleIncrement는 int32_t 유형이며, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 나타내며, 모터의 회전 방향은 이 매개변수의 부호에 의해 결정됩니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA7
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	위치 제어 하위 바이트	DATA[4] = *(uint8_t *)&angleIncrement
DATA[5]	위치 제어	DATA[5] = *((uint8_t *)&angleIncrement)+1
DATA[6]	위치 제어	DATA[6] = *((uint8_t *)&angleIncrement)+2
DATA[7]	위치 제어 상위 바이트	DATA[7] = *((uint8_t *)&angleIncrement)+3

비고:

- 이 명령에서 모터의 최대 속도는 상위 컴퓨터의 Max Speed 값에 의해 제한됩니다.
 - 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.
 - 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값으로 제한됩니다.
- MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값에 의해 제한됩니다.

드라이버 응답

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 2 읽기 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA7입니다).

17. 증분 위치 패쇄 제어 명령 2

호스트는 이 명령을 전송하여 모터의 위치 증분을 제어합니다.

1. 제어값 angleIncrement는 int32_t 타입이며, 실제 위치는 0.01degree/LSB에 해당합니다. 즉, 36000은 360°를 나타내며, 모터 회전 방향은 이 매개변수의 부호에 의해 결정됩니다.
2. 제어값 maxSpeed는 모터 회전의 최대 속도를 제한하며, uint32_t 유형으로 실제 회전 속도 1dps/LSB에 해당합니다. 즉, 360는 360dps를 나타냅니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xA8
DATA[1]	NULL	0x00
DATA[2]	속도 제한 하위 바이트	DATA[2] = *(uint8_t *)&maxSpeed
DATA[3]	속도 제한 상위 바이트	DATA[3] = *((uint8_t *)&maxSpeed)+1
DATA[4]	위치 제어 하위 바이트	DATA[4] = *(uint8_t *)&angleIncrement
DATA[5]	위치 제어	DATA[5] = *((uint8_t *)&angleIncrement)+1
DATA[6]	위치 제어	DATA[6] = *((uint8_t *)&angleIncrement)+2
DATA[7]	위치 제어 상위 바이트	DATA[7] = *((uint8_t *)&angleIncrement)+3

비고:

1. 이 제어 모드에서 모터의 최대 가속도는 상위 컴퓨터의 Max Acceleration 값에 의해 제한됩니다.
2. 이 제어 모드에서 MF, MH, MG 모터의 최대 토크 전류는 상위 컴퓨터의 Max Torque Current 값으로 제한됩니다; MS 모터의 최대 출력은 상위 컴퓨터의 Max Power 값으로 제한됩니다.

드라이버 응답

모터는 명령 수신 후 호스트에 응답합니다. 모터 응답 데이터는 모터 상태 2 읽기 명령과 동일합니다(명령 바이트 DATA[0]만 다르며, 여기서는 0xA8입니다).

18. 제어 매개변수 읽기 명령

호스트가 이 명령을 전송하여 현재 모터의 제어 매개변수를 읽습니다. 읽는 매개변수는 번호 controlParamID로 결정되며, [모터 제어 매개변수 참조](#)

[참조하십시오.](#)

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xC0
DATA[1]	제어 매개변수 번호	DATA[1] = controlParamID
DATA[2]	NULL	DATA[2] = 0x00
DATA[3]	NULL	DATA[3] = 0x00
DATA[4]	NULL	DATA[4] = 0x00
DATA[5]	NULL	DATA[5] = 0x00
DATA[6]	NULL	DATA[6] = 0x00
DATA[7]	NULL	DATA[7] = 0x00

드라이버 응답

드라이버 응답 데이터에는 읽는 매개변수 값이 포함되어 있으며, 구체적인 매개변수는 [모터 제어 매개변수 표](#)를 참조하십시오.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xC0
DATA[1]	제어 매개변수 번호	DATA[1] = controlParamID
DATA[2]	제어 매개변수 바이트 1	DATA[2] = controlParamByte1
DATA[3]	제어 매개변수 바이트 2	DATA[3] = 제어 매개변수 바이트 2
DATA[4]	제어 매개변수 바이트 3	DATA[4] = 제어 매개변수 바이트 3

DATA[5]	제어 매개변수 바이트 4	DATA[5] = 제어 매개변수 바이트 4
DATA[6]	제어 매개변수 바이트 5	DATA[6] = 제어 매개변수 바이트 5
DATA[7]	제어 매개변수 바이트 6	DATA[7] = 제어 매개변수 바이트 6

19. 제어 매개변수 쓰기 명령

호스트가 이 명령을 전송하면 제어 매개변수가 RAM에 즉시 적용되며, 전원 차단 시 무효화됩니다. 기록된 매개변수와 번호 controlParamID

[모터 제어 매개변수 표](#) 참조

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xC1
DATA[1]	제어 매개변수 번호	DATA[1] = controlParamID
DATA[2]	제어 매개변수 바이트 1	DATA[2] = controlParamByte1
DATA[3]	제어 매개변수 바이트 2	DATA[3] = 제어 매개변수 바이트 2
DATA[4]	제어 매개변수 바이트 3	DATA[4] = 제어 매개변수 바이트 3
DATA[5]	제어 매개변수 바이트 4	DATA[5] = 제어 매개변수 바이트 4
DATA[6]	제어 매개변수 바이트 5	DATA[6] = 제어 매개변수 바이트 5
DATA[7]	제어 매개변수 바이트 6	DATA[7] = 제어 매개변수 바이트 6

드라이버 응답

드라이버 응답 데이터에는 쓰기 후의 매개변수 값이 포함되어 있으며, 구체적인 매개변수는 [모터 제어 매개변수 표](#)를 참조하십시오

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0xC1
DATA[1]	제어 매개변수 번호	DATA[1] = controlParamID
DATA[2]	제어 매개변수 바이트 1	DATA[2] = controlParamByte1
DATA[3]	제어 매개변수 바이트 2	DATA[3] = 제어 매개변수 바이트 2
DATA[4]	제어 매개변수 바이트 3	DATA[4] = 제어 매개변수 바이트 3
DATA[5]	제어 매개변수 바이트 4	DATA[5] = 제어 매개변수 바이트 4
DATA[6]	제어 매개변수 바이트 5	DATA[6] = 제어 매개변수 바이트 5
DATA[7]	제어 매개변수 바이트 6	DATA[7] = 제어 매개변수 바이트 6

참고: 제어 매개변수 및 번호 설명은 제어 매개변수 읽기 명령 참고 사항 참조

20. 모터 인코더 데이터 읽기 명령

호스트가 이 명령을 전송하여 인코더의 현재 위치를 읽음

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x90
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 매개변수가 포함됩니다.

1. 인코더 위치 encoder(uint16_t 유형, 14비트 인코더의 수치 범위 0~16383)는 인코더 원시 위치에서 인코더 제로 오프셋을 뺀 값입니다.
2. 인코더 원시 위치 encoderRaw(uint16_t 유형, 14비트 인코더의 수치 범위 0~16383).
3. 인코더 오프셋 encoderOffset(uint16_t 타입, 14비트 인코더의 수치 범위 0~16383), 이 지점은 모터 각도의 0점으로 사용됩니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x90
DATA[1]	NULL	0x00
DATA[2]	인코더 위치 하위 바이트	DATA[2] = *(uint8_t *)&encoder
DATA[3]	인코더 위치 상위 바이트	DATA[3] = *((uint8_t *)&encoder)+1
DATA[4]	인코더 원시 위치 하위 바이트	DATA[4] = *(uint8_t *)&encoderRaw
DATA[5]	인코더 원시 위치 상위 바이트	DATA[5] = *((uint8_t *)&encoderRaw)+1
DATA[6]	인코더 제로 오프셋 하위 바이트	DATA[6] = *(uint8_t *)&encoderOffset
DATA[7]	인코더 제로 오프셋 상위 바이트	DATA[7] = *((uint8_t *)&encoderOffset)+1

21. 현재 위치를 ROM에 설정하여 모터 제로 포인트 명령으로 지정

모터 현재 위치의 인코더 원시값을 모터 전원 공급 후 초기 제로 포인트로 설정

- 이 명령은 전원을 재공급한 후에만 적용됩니다
- 이 명령은 제로 포인트를 드라이버의 ROM에 기록하며, 반복 기록은 칩 수명에 영향을 미칠 수 있으므로 빈번한 사용을 권장하지 않습니다

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x19
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 데이터 중 encoderOffset은 설정된 0 오프셋 값입니다

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x19
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	인코더 제로 오프셋 하위 바이트	DATA[6] = *(uint8_t *)&encoderOffset
DATA[7]	인코더 제로 오프셋 상위 바이트	DATA[7] = *((uint8_t *)&encoderOffset)+1

22. 다중 회전 각도 읽기 명령

호스트가 현재 모터의 다중 회전 절대 각도 값을 읽기 위해 이 명령을 전송함

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x92
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00

DATA[7]	NULL	0x00
---------	------	------

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 매개변수가 포함됩니다.

1. 모터 각도 motorAngle: int64_t 형식 데이터로, 양수는 시계 방향 누적 각도를, 음수는 반시계 방향 누적 각도를 나타내며 단위는 0.01° /LSB입니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x92
DATA[1]	각도 하위 바이트 1	DATA[1] = *((uint8_t *)&motorAngle)
DATA[2]	각도 바이트 2	DATA[2] = *((uint8_t *)&motorAngle)+1
DATA[3]	각도 바이트 3	DATA[3] = *((uint8_t *)&motorAngle)+2
DATA[4]	각도 바이트 4	DATA[4] = *((uint8_t *)&motorAngle)+3
DATA[5]	각도 바이트 5	DATA[5] = *((uint8_t *)&motorAngle)+4
DATA[6]	각도 바이트 6	DATA[6] = *((uint8_t *)&motorAngle)+5
DATA[7]	각도 바이트 7	DATA[7] = *((uint8_t *)&motorAngle)+6

23. 단일 회전 각도 읽기 명령

호스트가 현재 모터의 단일 회전 각도를 읽기 위해 이 명령을 전송합니다

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x94
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	NULL	0x00
DATA[5]	NULL	0x00
DATA[6]	NULL	0x00
DATA[7]	NULL	0x00

드라이버 응답

모터는 명령을 수신한 후 호스트에 응답하며, 이 프레임 데이터에는 다음 매개변수가 포함됩니다.

1. 모터 단일 회전 각도 circleAngle은 uint32_t 유형 데이터로, 인코더 제로점을 시작점으로 시계 방향으로 증가하며, 다시 제로점에 도달하면 값이 0으로 돌아갑니다. 단위는 0.01° /LSB이며, 값 범위는 0~36000*감속비-1입니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x94
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	단일 회전 각도 하위 바이트 1	DATA[4] = *((uint8_t *)&circleAngle)
DATA[5]	단일 회전 각도 바이트 2	DATA[5] = *((uint8_t *)&circleAngle)+1
DATA[6]	단일 회전 각도 바이트 3	DATA[6] = *((uint8_t *)&circleAngle)+2
DATA[7]	단일 회전 각도 상위 바이트 4	DATA[7] = *((uint8_t *)&circleAngle)+3

24. 현재 위치를 임의 각도로 설정(RAM에 쓰기)

호스트가 이 명령을 전송하여 모터의 현재 위치를 임의의 각도로 설정합니다. 다중 회전 각도 값 motorAngle은 int32_t 유형 데이터이며, 데이터 단위는 0.01° /LSB입니다.

데이터 영역	설명	데이터
DATA[0]	명령 바이트	0x95
DATA[1]	NULL	0x00
DATA[2]	NULL	0x00
DATA[3]	NULL	0x00
DATA[4]	다중 회전 각도 하위 바이트 1	DATA[4] = *((uint8_t *)&motorAngle)

DATA[5]	다중 회전 각도 바이트 2	DATA[5] = *((uint8_t *)& motorAngle)+1)
DATA[6]	다중 회전 각도 바이트 3	DATA[6] = *((uint8_t *)& motorAngle)+2)
DATA[7]	다중 회전 각도 상위 바이트 4	DATA[7] = *((uint8_t *)& motorAngle)+3)

드라이버 응답

모터는 명령 수신 후 호스트에 응답하며, 프레임 데이터는 호스트가 전송한 것과 동일

부록 1: 모터 제어 매개변수 표

모터 제어 매개변수 표	
매개변수 번호 ParamID	제어 매개변수 설명
10 (0x0A)	각도 루프 PID, 세 가지 매개변수 포함 anglePidKp (각도 루프 kp, uint16_t 형식) controlParamByte1 = *((uint8_t *)& anglePidKp) controlParamByte2 = *((uint8_t *)& anglePidKp)+1) anglePidKi (각도 루프 Ki, uint16_t 유형) controlParamByte3 = *((uint8_t *)& anglePidKi) controlParamByte4 = *((uint8_t *)& anglePidKi)+1) anglePidKd(각도 루프 kd, uint16_t 타입) controlParamByte5 = *((uint8_t *)& anglePidKd) controlParamByte6 = *((uint8_t *)& anglePidKd)+1)
11 (0x0B)	속도 루프 PID, 세 개의 매개변수 포함 speedPidKp (속도 루프 kp, uint16_t 유형) controlParamByte1 = *((uint8_t *)& speedPidKp) controlParamByte2 = *((uint8_t *)& speedPidKp)+1) speedPidKi (속도 루프 Ki, uint16_t 유형) controlParamByte3 = *((uint8_t *)& speedPidKi) controlParamByte4 = *((uint8_t *)& speedPidKi)+1) speedPidKd(속도 루프 kd, uint16_t 타입) controlParamByte5 = *((uint8_t *)& speedPidKd) controlParamByte6 = *((uint8_t *)& speedPidKd)+1)
12 (0x0C)	전류 루프 PID, 세 개의 매개변수 포함 currentPidKp (전류 루프 kp, uint16_t 타입) controlParamByte1 = *((uint8_t *)& currentPidKp) controlParamByte2 = *((uint8_t *)& currentPidKp)+1) currentPidKi (전류 루프 Ki, uint16_t 유형) controlParamByte3 = *((uint8_t *)& currentPidKi) controlParamByte4 = *((uint8_t *)& currentPidKi)+1) currentPidKd(전류 루프 kd, uint16_t 타입) controlParamByte5 = *((uint8_t *)& currentPidKd) controlParamByte6 = *((uint8_t *)& currentPidKd)+1)
30(0x1E)	inputTorqueLimit(최대 토크 전류, int16_t 타입) controlParamByte3 = *((uint8_t *)& inputTorqueLimit) controlParamByte4 = *((uint8_t *)& inputTorqueLimit)+1)
32 (0x20)	inputSpeedLimit(최대 속도, int32_t 유형) controlParamByte3 = *((uint8_t *)& inputSpeedLimit) controlParamByte4 = *((uint8_t *)& inputSpeedLimit)+1)
	controlParamByte5 = *((uint8_t *)& inputSpeedLimit)+2) controlParamByte6 = *((uint8_t *)& inputSpeedLimit)+3)

34 (0x22)	inputAngleLimit(각도 제한, int32_t 형식) controlParamByte3 = *(uint8_t *)& inputAngleLimit controlParamByte4 = *((uint8_t *)& inputAngleLimit)+1 controlParamByte5 = *((uint8_t *)& inputAngleLimit)+2) controlParamByte6 = *((uint8_t *)& inputAngleLimit)+3)
36 (0x24)	inputCurrentRamp(전류 경사도, int32_t 형식) controlParamByte3 = *(uint8_t *)& inputCurrentRamp controlParamByte4 = *((uint8_t *)& inputCurrentRamp)+1 controlParamByte5 = *((uint8_t *)& inputCurrentRamp)+2) controlParamByte6 = *((uint8_t *)& inputCurrentRamp)+3)
38 (0x26)	inputSpeedRamp (속도 경사, int32_t 타입) controlParamByte3 = *(uint8_t *)& inputSpeedRamp controlParamByte4 = *((uint8_t *)& inputSpeedRamp)+1 controlParamByte5 = *((uint8_t *)& inputSpeedRamp)+2) controlParamByte6 = *((uint8_t *)& inputSpeedRamp)+3)